

View recent browsing across windows and devices

CourseNew TabGitHub DashboardView 1 - @perwangelmed12's uNew Tab

mitaoe.codedtantra.com/secure/course.jsp?eudd=696c54060aba96214c8b7d5ee/contents

80%00%

FPS 84

CODETANTRA

HomeLearn Anywhere

202501090044@mitaoe.ac.inSupportLogout

Essentials of Data Science Laboratory - 2304102L - 2026 - 2304102L

DashboardContentsCalendarAssessmentsSyllabus

Search course

ctrl + k

1. Practical 1

2. Practical 2

3. Practical 3

4. Practical 4

5. Practical 5

97%
Course progress

0%100%

100% units left

In Progress

Pickup from where you left off!

Scatter Plot for Age vs. Fare by Survived

Practical 5 • Lab Assignment

Resume

Practical 1

Unit • 100% completed

Practical 2

Unit • 100% completed

Practical 3

Unit • 100% completed

Practical 4

Unit • 90% completed

Practical 5

Unit • 100% completed

View recent browsing across windows and devices

Course

New Tab

GitHub Dashboard

View 1 - @penwagimad12's u

New Tab

FPS 100

mitaace.codetantra.com/secure/course.jsp?euid=696c54060aba96214c8b7d5e

00%

Support

Logout

CODETANTRA

Home

Learn Anywhere

202501090044@mitaace.ac.in

Support

Logout

Essentials of Data Science Laboratory - 2304102L - 2026 - 2304102L

Dashboard

Contents

Calendar

Assessments

Syllabus

97%

Course progress

0%100%

Completed

In Progress

Resume

May

April

September

October

November

December

January

February

March

April

May

← Prev

Next →

Less

5.00

4.00

3.00

More

Description

Essentials of Data Science Laboratory - 2304102L

Upcoming tests

No upcoming exams in the next 7 days

Instructors

Supriya Sabale

Credits

Information Unavailable

Course Outcomes

Information Unavailable

Course

Course

New Tab

GitHub Dashboard

View 1 - @pewangalmeid12's u X

New Tab

FPS 53

← → ↺

mitaoe.codedtantra.com/secure/course.jsp?eucdd=696c54060aba96214c8b7d5e4/contents/696c54060aba96214c8b7d5d/67e2a5387c0b912d660eba29

00%

202501090044@mitaoe.ac.in

Support

Logout

CODETANTRA

Home

Learn Anywhere

5.2.11. Scatter Plot for Age vs. Fare by Survived

Write a Python code to plot a scatter plot showing the relationship between the 'Age' and 'Fare' columns in the Titanic dataset, with points color-coded by survival status. The scatter plot should display the following specifications:
1. Use the **Age** column for the x-axis and the **Fare** column for the y-axis.
2. Color the points based on the **Survived** column: Red for passengers who did not survive (Survived = 0); Blue for passengers who survived (Survived = 1).
3. Set the title of the plot to **"Age vs. Fare by Survival"**.
4. Label the x-axis as **"Age"** and the y-axis as **"Fare"**.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Bramson, Mr. Owen Harris", male, 21,1,0,0, 11171, 7.25, , S
3,1,1,"Combs, Mrs. John Bradley (Florence Briggs Tappan)", female, 38,1,0, 17509, 71.2833, C85, Q
4,1,3,"Mackinnon, Miss. Laina", female, 28,0,0, 51067, 53.1, S
5,0,3,"Patricola, Mrs. Jacques Heath (Lily May Peel)", female, 25,1,0, 112802, 51.2, C223, S
6,0,3,"Allen, Mr. William Henry", male, 35,0,0, 873498, 8.65, , S
7,0,3,"Moran, Mr. James", male, 19,0, 10437, 0, Q
8,0,1,"McCarthy, Mr. Timothy J", male, 54,0,0, 17463, 51.6625, E46, S
9,0,3,"Palsson, Master. Gosta Leonard", male, 2,3,1, 349909, 31.875, , S
10,1,3,"Jansson, Mrs. Oscar W (Ellisabeth Vilhelmina Berg)", female, 27,0,1, 14742, 11.1333, , S
16,1,2,"Nasser, Mrs. Nicholas (Adele Achen)", female, 14,1,0, 237736, 30.4708, , C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

AgeFareS...

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 # Write your code here for Scatter Plot for Age vs. Fare by Survived
17 # Write your code here for Scatter Plot for Age vs. Fare by Survived
18 colors = data['Survived'].map([0:'red',1:'blue'])
19 plt.scatter(data['Age'], data['Fare'], c=colors)
20 plt.title('Age vs. Fare by Survival')
21 plt.xlabel('Age')
22 plt.ylabel('Fare')
23 plt.show()
24
25
```

Average time: 0.465 s, 465.00 ms

Maximum time: 0.465 s, 465.00 ms

1 out of 1 shown test case(s) passed

Test case 1

Expected output

Actual output

Terminal

Test Cases

Prev

Reset

Submit

Course

Course

New Tab

GitHub Dashboard

View 1 - @penwainmed123 u X

New Tab

FPS 26

initiaioe.codetantra.com/secure/course.jpg?eudd=696c54060aba96214c8b7d5e4/contents/696c54060aba96214c8b7d5d/67e2a2977cd0b912d060eb621

00%

202501090044@initiaioe.ac.in Support Logout

DEETANTRA Home Learn Anywhere

5.2.18. Scatter Plot for Age vs. Fare

Write a Python code to plot a scatter plot showing the relationship between the 'Age' and 'Fare' columns in the Titanic dataset. The scatter plot should display the following specifications:
1. Use the Age column for the x-axis and the Fare column for the y-axis.
2. Set the title of the plot to "Age vs. Fare".
3. Label the x-axis as "Age" and the y-axis as "Fare".

The Titanic dataset contains columns as shown below.

Passenger Id	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1, 0, 3, "Braund, Mr. Owen Harris", male, 22, 1, 0, A/5 21171, 7.25, S
2, 1, 1, "Cumings, Mrs. John Bradley (Florence Briggs Thayer)", female, 38, 1, 0, PC 17599, 71.2833, C85, C
3, 1, 3, "Heikkinen, Miss. Laina", female, 26, 0, 0, STON/O2, 51.8625, S
4, 1, 1, "Nutville, Mrs. Jacques Heath (Lily May Peel)", female, 31, 1, 0, 113802, 53.1, C123, S
5, 0, 3, "Allen, Mr. William Henry", male, 35, 0, 0, 373450, 8.05, S
6, 0, 3, "Moran, Mr. James", male, 19, 0, 1, 330871, 5.40, Q
7, 0, 1, "McCarthy, Mr. Timothy J", male, 54, 0, 0, 17461, 51.8625, E46, S
8, 0, 3, "Palsson, Master. Gosta Leonard", male, 2, 3, 1, 349909, 21.075, S
9, 1, 3, "Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)", female, 27, 0, 1, 167042, 31.1733, S
10, 1, 4, "Nasser, Mrs. Nicholas (Adele Achem)", female, 14, 1, 0, 237736, 26.0786, C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

AgeFare5...

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 # Write your code here for Box Plot for Fare by Pclass
17
18 # Write your code here for Box Plot for Fare by Pclass
19 plt.scatter(data['Age'], data['fare'])
20 plt.title('Age vs. Fare')
21 plt.xlabel('Age')
22 plt.ylabel('fare')
23 plt.show()
24
```

Average time: 0.914 s, 154.00 ms; Maximum time: 0.914 s, 154.00 ms; 1 out of 1 shown test case(s) passed

Test case 1: Expected output (Age vs. Fare scatter plot) and Actual output (Age vs. Fare scatter plot)

Terminal, Test Cases

Prev, Report, Submit, Next

Course

Course

New Tab

GitHub Dashboard

View | @perwainkred12's u X

New Tab

initiatecodetantra.com/secure/course.jpg?eudd=696c54060aba96214c8b7d5e7/contents/696c54060aba96214c8b7d5d/696c54060aba96214c8b7d5d/67e2a1727cd0b912d660eb339

CODETANTRA

Home

Learn Anywhere

202301090044@mitfoc.ac.inSupportLogout

5.2.9. Box Plot for Fare by Pclass

Write a Python code to plot a boxplot that shows the distribution of the 'Fare' column from the Titanic dataset based on the passenger class (Pclass). The boxplot should display the following specifications:

1. Use the Pclass column to group the data for the boxplot.

2. Set the title of the plot to "Fare by Pclass".

3. Remove the default subtitle with plt.suptitle("").

4. Label the x-axis as 'Pclass' and the y-axis as 'Fare'.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,1,"Brands, Mr. Owen Harris", male, 22,1,0,0,5, 21171,7.25,S
2,1,1,"Cunha, Mrs. John Bradley (Florence Briggs Thayer)", female, 38,1,0,PC 17599,71.2833,C85,C
3,1,1,"Heikkinen, Miss. Laina", female, 26,0,0,STON/O2, 3181262,7.925,S
4,1,1,"Nesbitt, Mrs. Jacobus Heath (Lily May Peel)", female, 25,1,0,312802,53.1,C22,S
5,0,1,"Allen, Mr. William Henry", male, 35,0,0,373498,8.65,S
6,0,1,"Mann, Mr. James", male, 27,0,0,310127,5.42,S,Q
7,0,1,"McCarthy, Mr. Timothy J", male, 54,0,0,17463,51.6625,C40,S
8,0,1,"Palsson, Master. Gosta Leonard", male, 2,3,1,349909,31.875,S
9,1,1,"Jansson, Mrs. Oscar W (Ellisabeth Vilhelmina Berg)", female, 27,0,1,14742,31.3333,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)", female, 14,1,0,237736,30.4708,C

Note: Refer to the visible test case for better reference.

Sample Test Cases

BoxPlotF...

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

import pandas as pd

import matplotlib.pyplot as plt

Load the Titanic dataset

data = pd.read_csv('Titanic-Dataset.csv')

Data Cleaning

data['Age'].fillna(data['Age'].median(), inplace=True)

data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)

data.drop('Cabin', axis=1, inplace=True)

Convert categorical features to numeric

data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

Write your code here for Box Plot for Fare by Pclass

import pandas as pd

import matplotlib.pyplot as plt

Load the Titanic dataset

data = pd.read_csv('Titanic-Dataset.csv')

Data Cleaning

data['Age'].fillna(data['Age'].median(), inplace=True)

data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)

data.drop('Cabin', axis=1, inplace=True)

Convert categorical features to numeric

data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

Box Plot: Fare by Pclass

Average time

0.539 s

109.00 ms

Maximum time

0.539 s

109.00 ms

1 out of 1 shown test case(s) passed

Test case 1

Expected output

Actual output

Fare by Pclass

Fare by Pclass

Terminal

Test Cases

Prev

Reset

Submit

Next

Course

Course

New Tab

GitHub Dashboard

View | Epenwainvred12's u X

New Tab

FPS 114

← → ↺

mitace.codedtantra.com/secure/course.php?eudd=696c54060aba96214c8b7d5e7/contents/696c54060aba96214c8b7d5d/67e29d57c0b912d860eab43

00%

202501090044@mitase.ac.in

Support

Logout

DEETANTRA

Home

Learn Anywhere

5.2.B. Box Plot for Age by Survived

0.03s

🔍

📄

🔗

🔄

Write a Python code to plot a boxplot that shows the distribution of the 'Age' column from the Titanic dataset based on whether passengers survived or not. The boxplot should display the following specifications:
1. Use the Survived column to group the data for the boxplot (0 = Did not survive, 1 = Survived).
2. Set the title of the plot to "Age by Survival".
3. Remove the default subtitle with plt.suptitle("").
4. Label the x-axis as 'Survived' and the y-axis as 'Age'.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Bramson, Mr. Owen Harris", male, 21,1,0,0,5 21171,7.25,S
2,1,1,"Combs, Mrs. John Bradley (Florence Briggs Tappan)", female, 38,1,0,PC 17599,71.2833,C85,C
3,1,1,"Heikkinen, Miss. Laina", female, 26,0,0,STON/O2 3101282,7.925,S
4,1,1,"Pettersen, Mrs. Jacobus Math (Lily May Peel)", female, 25,1,0,312862,53.2,C223,S
5,0,3,"Allen, Mr. William Henry", male, 35,0,0,873498,8.65,S
6,0,3,"Moran, Mr. James", male, 14,0,1,516877,9.45,S,Q
7,0,1,"McCarthy, Mr. Timothy J", male, 54,0,0,17463,51.6625,C40,S
8,0,3,"Palsson, Master. Gosta Leonard", male, 2,3,1,349909,31.875,S
9,1,3,"Jansson, Mrs. Oscar W (Ellisabeth Vilhelmina Berg)", female, 27,0,1,14742,33.3333,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)", female, 14,1,0,237736,30.4708,C

Note: Refer to the visible test case for better reference.

Sample Test Cases

4

BoxPlotF...

0.03s

Submit

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

import pandas as pd

import matplotlib.pyplot as plt

Load the Titanic dataset

data = pd.read_csv('Titanic-Dataset.csv')

Data Cleaning

data['Age'].fillna(data['Age'].median(), inplace=True)

data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)

data.drop('Cabin', axis=1, inplace=True)

Convert categorical features to numeric

data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

import pandas as pd

import matplotlib.pyplot as plt

Load the Titanic dataset

data = pd.read_csv('Titanic-Dataset.csv')

Data Cleaning

data['Age'].fillna(data['Age'].median(), inplace=True)

data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)

data.drop('Cabin', axis=1, inplace=True)

Convert categorical features to numeric

data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

Box Plot for Age by Survived

plt.figure(figsize=(8, 6))

data.boxplot(column='Age', by='Survived')

Average time

0.414 s

414.00 ms

Maximum time

0.414 s

414.00 ms

1 out of 1 shown test case(s) passed

Test case 1

0.03s

0.03s

0.03s

0.03s

Expected output

Actual output

Age by Survival

Age by Survival

Terminal

Test Cases

Prev

Next

Submit

Reset

Course

Course

New Tab

GitHub Dashboard

View 1 - @pawangakwad12's u X

New Tab

FPS 85

mitaoe.coodetantra.com/secure/course.jsp?eudd~698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d5e/67c2992e7c0b912d660ea071

80%

202501795044@mitaoe.ac.in

Support

Logout

5.2.7. Box plot for Age Distribution

Write a Python code to plot a boxplot that shows the distribution of the 'Age' column from the Titanic dataset across different passenger classes. The boxplot should display the following specifications:
1. Use the Pclass column to group the data for the boxplot.
2. Set the title of the plot to "Age by Pclass"
3. Remove the default subtitle with plt.suptitle("")
4. Label the x-axis as 'Pclass' and the y-axis as 'Age'.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,1,"Brando, Mr. Dean Harris",male,22,1,0,0,5 21371,7.25,S
2,1,1,"Carpenter, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17569,71.2835,C85,C
3,1,3,"McKAY, Mrs. Miss. Lila",female,28,0,0,STON/O2, 3182282,9.005,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,1,"Allen, Mr. William Henry",male,35,0,0,373458,8.05,S
6,0,2,"Morris, Mr. James",male,40,0,0,159677,8.4580,Q
7,0,1,"McCortney, Mr. Timothy J",male,54,0,0,17461,51.8625,146,S
8,0,3,"Falconer, Master. Gust Leonard",male,2,3,1,045008,11.075,S
9,1,3,"Johnson, Mrs. Oscar W (Ellenesth Vilhelmina Berg)",female,27,0,2,247742,11.1355,S
10,1,2,"Masser, Mrs. Nicholas (Adele Achen)",female,14,1,0,277756,10.9708,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

BoxPlotF...

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15 # Write your code here for Box Plot for Age by Pclass
16
17 import pandas as pd
18 import matplotlib.pyplot as plt
19
20 # Load the Titanic dataset
21 data = pd.read_csv('Titanic-Dataset.csv')
22
23 # Data Cleaning
24 data['Age'].fillna(data['Age'].median(), inplace=True)
25 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
26 data.drop('Cabin', axis=1, inplace=True)
27
28 # Convert categorical features to numeric
29 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
30 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
31
32 # Box Plot for Age by Pclass
33 plt.figure(figsize=(8, 6))
34 data.boxplot(column='Age', by='Pclass')
```

Average time: 0.557 s, Maximum time: 0.557 s, 1 out of 4 shown test case(s) passed

Test case 1: Expected output: Actual output:

Terminal Test cases

Prev Next Submit

Course

Course

New Tab

GitHub Dashboard

View 1 - @pawongaknoid12's u

New Tab

FPS 85

mitaoe.codetantra.com/secure/course.jsp?seudd=698c54860aba96214c6b7d3e#/content/098c54860aba96214c6b7d6b/698c54860aba96214c6b7d6d/67e297bf7c0b912d860a9c81

CODETANTRA

2025010950344@mitaoe.ac.in

Support

Logout

5.2.6. Bar Plot for Survival by Embarked

Write a Python code to plot a stacked bar chart showing the survival count for passengers based on their embarkation location in the Titanic dataset. The chart should display the following specifications.

- Use the Embarked column to determine the embarkation location. After converting this column into dummy variables (using pd.get_dummies()), plot the survival count based on the Embarked_Q column (representing passengers who embarked from Queenstown) in relation to survival.
- Set the chart type to 'bar' and make it stacked.
- Add the title "Survival by Embarked" to the chart.
- Label the x-axis as "Embarked" and the y-axis as "Count".
- Include a legend to distinguish between survivors and non-survivors (label the legend as "Survived" and "Not Survived").

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,0,0,53.1,7.25,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.0,83,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2, 3101282,7.92,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,1,"Allen, Mr. William Henry",male,35,0,0,01506,8.45,S
6,0,3,"Morris, Mr. Samuel",male,40,0,0,09577,5.42,S
7,0,1,"McCarthy, Mr. Timothy J",male,34,0,0,1346,51.8625,C46,S
8,0,1,"Palsson, Nector (Gusta Leonard)",male,2,1,1,100969,21.075,S
9,1,3,"Johnson, Mrs. Oscar W (Elizabeth Vilhelmina Berg)",female,47,0,2,347742,51.133,S
10,1,2,"Nasser, Mrs. Nicholas (Aminé Aknos)",female,34,1,0,217736,18.0796,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

BarPlotOf...

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv("Titanic-Dataset.csv")
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15
16 # Write your code here for Bar Plot for Survival by Embarked
17 # Write your code here for Bar Plot for Survival by embarked
18 import pandas as pd
19 import matplotlib.pyplot as plt
20
21 # Load the Titanic dataset
22 data = pd.read_csv("Titanic-Dataset.csv")
23
24 # Data Cleaning
25 data['Age'].fillna(data['Age'].median(), inplace=True)
26 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
27 data.drop('Cabin', axis=1, inplace=True)
28
29 # Convert categorical features to numeric
30 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
31 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
32
33 # Write your code here for Bar Plot for Survival by Embarked
34
```

Average time

0.624 s

624.00 ms

Maximum time

0.624 s

624.00 ms

1 out of 1 shown test case(s) passed

Test case 1

Expected output

Actual output

Survived by Embarked

Survived by Embarked

Terminal

Test cases

Prev

Reset

Submit

Next

Course

Course

New Tab

GitHub Dashboard

View 1 · @pawangknad12's u...

New Tab

miratae.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d6d/67e291be7c0b912d860e876f

CODETANTRA

202501090044@miratae.ac.in

Support

Logout

6.2.5. Bar Plot for Survival by Pclass

Write a Python code to plot a stacked bar chart that shows the count of passengers who survived and did not survive, grouped by passenger class (Pclass), in the Titanic dataset. The chart should display the following specifications:

1. Group the data by the Pclass column and count the number of survivors (0 = Did not survive, 1 = Survived) for each class using value_counts()

2. Use a stacked bar chart to display the survival counts.

3. Add the title "Survival by Pclass" to the chart.

4. Label the x-axis as 'Pclass' and the y-axis as 'Count'.

5. The legend should indicate 'Not Survived' and 'Survived'.

The Titanic dataset contains columns as shown below.

Passenger Id	Survived	Pclass	Name	Sex	Age	SibSp	ParCh	Ticket	Fare	Cabin	Embarked
1	0	3	Brands, Mr. Owen Harris	male	22	1	0	AP/ 21571	7.25	5	
2	1	1	Cooking, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17596	51.2832	C85	C
3	1	3	Swickins, Miss. Laina	female	20	0	0	STON/O2	53.00	5	
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113801	53.1	C123	5
5	0	3	Allan, Mr. William Henry	male	29	0	0	373450	8.45	5	
6	0	3	Morris, Mr. James	male	0	0	0	538077	0.4586	Q	
7	0	1	McCarthy, Mr. Timothy J	male	54	0	0	17564	51.8625	E84	5
8	0	1	Palsson, Master. Gosta Leonard	male	2	1	1	349909	21.075	5	
9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27	0	2	547742	31.1032	5	
10	1	2	Nasser, Mrs. Nicholas (Adelie Achen)	female	34	1	0	237736	30.0708	C	

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, ParCh, Ticket, Fare, Cabin, Embarked

1,0,3,"Brands, Mr. Owen Harris",male,22,1,0,AP/ 21571,7.25,5

2,1,1,"Cooking, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17596,51.2832,C85,C

3,1,3,"Swickins, Miss. Laina",female,20,0,0,STON/O2, 53.00,5

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113801,53.1,C123,5

5,0,3,"Allan, Mr. William Henry",male,29,0,0,373450,8.45,5

6,0,3,"Morris, Mr. James",male,0,0,538077,0.4586,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17564,51.8625,E84,5

8,0,1,"Palsson, Master. Gosta Leonard",male,2,1,1,349909,21.075,5

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,547742,31.1032,5

10,1,2,"Nasser, Mrs. Nicholas (Adelie Achen)",female,34,1,0,237736,30.0708,C

Note:

Refer to the visible test case for better reference.

Ensure you use the `groupby()` function with `value_counts()` to count the survivors and non-survivors for each Pclass.

Do not manually use `size()` or `unstack()` without `value_counts()`. Use the `value_counts()` method for counting survival status directly.

Sample Test Cases

BarPlotOf...

1

import pandas as pd

2

import matplotlib.pyplot as plt

3

4

Load the Titanic dataset

5

data = pd.read_csv('Titanic-Dataset.csv')

6

7

Data Cleaning

8

data['Age'].fillna(data['Age'].median(), inplace=True)

9

data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)

10

data.drop('Cabin', axis=1, inplace=True)

11

12

Convert categorical features to numeric

13

data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

14

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

15

16

Write your code here for Bar Plot for Survival by Pclass

17

survival_by_class = data.groupby('Pclass')['Survived'].value_counts().unstack().fillna(0)

18

survival_by_class.columns = ['Not Survived', 'Survived']

19

survival_by_class.plot(kind='bar', stacked=True)

20

plt.title('Survival by Pclass')

21

plt.xlabel('Pclass')

22

plt.ylabel('Count')

23

plt.legend(title=None)

24

plt.show()

25

26

27

28

29

Average time

0.586 s

586.00 ms

Maximum time

0.586 s

586.00 ms

1 out of 1 shown test case(s) passed

Test case 1

600 ms

Debug

Test Cases

Expected output

Actual output

Survival by Pclass

Survival by Pclass

Terminal

Test Cases

Prev

Reset

Submit

Next

Course

Course

New Tab

GitHub Dashboard

View 1 · @penwagained12's u X

New Tab

FPS 23

milade.codetantra.com/secure/course.jsp?eudd=696c54060aba96214c8b7d5e4/contents/696c54060aba96214c8b7d5d/67c28fb27c0b912d860e7db4

CODETANTRA

Home

Learn Anywhere

202501090144@mittos.ac.in

Support

Logout

1.2.4. Bar Plot for Survival by Gender

Write a Python code to plot a stacked bar chart that shows the count of passengers who survived and did not survive, grouped by gender. In the Titanic dataset, The chart should display the following specifications:
1. Group the data by the 'Sex' column, then use the value_counts() function to count the occurrences of survivors (0 = Did not survive, 1 = Survived) for each gender.
2. Use a stacked bar chart to display the survival counts.
3. Add the title "Survival by Gender" to the chart.
4. Label the x-axis as "Gender" and the y-axis as "Count".
5. The legend should indicate "Not Survived" and "Survived".

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Brown, Mr. Owen Harris	male	22	1	0	4/9	21571	7.25	S
2	1	1	Mr. Carlisle, Mrs. John Bradley (Mrs. Florence Briggs Thayer)	female	38	1	0	PC 17569	71.2833	CAS C	S
3	1	1	Mr. McKinnon, Miss. Lister	female	26	0	0	STON/O2	3101262	7.925	S
4	1	1	Mr. Fretwell, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	313861	53.1	C223 S	S
5	0	1	Mr. Miller, Mr. William Henry	male	35	0	0	373450	8.49	S	S
6	0	5	Mr. Moran, Mr. James	male	40	0	0	10957	3.455	Q	S
7	0	1	Mr. McCarthy, Mr. Timothy J.	male	54	0	0	17463	51.6625	E46 S	S
8	0	3	Mr. Palkson, Master. Gosta Leonard	male	2	1	1	349909	21.075	S	S
9	1	0	Mr. Johnson, Mrs. Oscar N (Elizabeth Valdemarne Berg)	female	27	0	1	247742	31.2323	S	S
10	1	1	Mr. Messer, Mrs. Nicholas (Adèle Achong)	female	34	1	0	237736	58.4786	C	S

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,Mr. Brown, Mr. Owen Harris,male,22,1,0,4/9,21571,7.25,S
2,1,1,Mr. Carlisle, Mrs. John Bradley (Mrs. Florence Briggs Thayer),female,38,1,0,PC 17569,71.2833,CAS C
3,1,1,Mr. McKinnon, Miss. Lister,female,26,0,0,STON/O2,3101262,7.925,S
4,1,1,Mr. Fretwell, Mrs. Jacques Heath (Lily May Peel),female,35,1,0,313861,53.1,C223 S
5,0,1,Mr. Miller, Mr. William Henry,male,35,0,0,373450,8.49,S
6,0,5,Mr. Moran, Mr. James,male,40,0,0,10957,3.455,Q
7,0,1,Mr. McCarthy, Mr. Timothy J.,male,54,0,0,17463,51.6625,E46 S
8,0,3,Mr. Palkson, Master. Gosta Leonard,male,2,1,1,349909,21.075,S
9,1,0,Mr. Johnson, Mrs. Oscar N (Elizabeth Valdemarne Berg),female,27,0,1,247742,31.2323,S
10,1,1,Mr. Messer, Mrs. Nicholas (Adèle Achong),female,34,1,0,237736,58.4786,C

Note: Refer to the visible test case for better reference.

Sample Test Cases

BarPlotOf...

1import pandas as pd
2import matplotlib.pyplot as plt
3
4# Load the Titanic dataset
5data = pd.read_csv('Titanic-Dataset.csv')
6
7# Data Cleaning
8data['Age'].fillna(data['Age'].median(), inplace=True)
9data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10data.drop('Cabin', axis=1, inplace=True)
11
12# Convert categorical features to numeric
13data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15# Write your code here for Bar Plot for Survival by Gender
16survival_by_gender = data.groupby('Sex')['Survived'].value_counts().unstack().fillna(0)
17survival_by_gender.columns = ['Not Survived', 'Survived']
18survival_by_gender.index = ['0', '1']
19survival_by_gender.plot(kind='bar', stacked=True)
20plt.title('Survival by Gender')
21plt.xlabel('Gender')
22plt.ylabel('Count')
23plt.legend(title=None)
24plt.show()
25
26
27
28
29
30
31
32
33
34

Average time0.632 s
0.632 s
0.62 s

Maximum time0.632 s
0.62 s
0.62 s

1 out of 1 shown test case(s) passed

Test case 10.62 sDebug

Expected output

Actual output

TerminalTest cases

Prev

Next

Submit

Next

View recent browsing across windows and devices

Course

New Tab

GitHub Dashboard

View 1 - @pewangalmed123 u X

New Tab

FPS 77

milatae.codedtantra.com/secure/course.jpg?eudd=696c54060aba96214c8b7d5e4/contents/696c54060aba96214c8b7d5d4/67e20d9e7c0b912d060c75ad

00%

202501090044@milatae.ac.in

Support

Logout

CODETANTRA

Home

Learn Anywhere

5.2.3. Bar plot of survival rate of passengers

Write a Python code to plot a bar chart that shows the count of passengers who survived and did not survive in the Titanic dataset. The chart should display the following specifications:
1. Use the 'Survived' column to show the count of survivors (0 = Did not survive, 1 = Survived).
2. Set the chart type to 'bar'.
3. Add the title 'Survival Count' to the chart.
4. Label the x-axis as 'Survived' and the y-axis as 'Count'.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Bramson, Mr. Owen Harris", male, 21,1,0,0, 21171, 7.25, S
2,1,1,"Cunha, Mrs. John Bradley (Florence Briggs Thayer)", female, 38,1,0, PC 17599, 71.2833, C85, S
3,1,3,"Mackinnon, Miss. Laina", female, 26,0,0, STON/O2, 3181282, 7.025, S
4,1,1,"Petrucci, Mrs. Jacques Heath (Lily May Peel)", female, 25,1,0, 312802, 53.1, C223, S
5,0,3,"Allen, Mr. William Henry", male, 35,0,0, 873406, 8.65, S
6,0,3,"Hansen, Mr. Joesph", male, 44,4, 138027, 9.45, Q
7,0,1,"McCarthy, Mr. Timothy J", male, 54,0,0, 17463, 51.0, S221, S40, S
8,0,3,"Paisson, Master. Roctus Leonard", male, 2,3,1, 349909, 21.075, S
9,1,3,"Jansson, Mrs. Oscar W (Ellisabeth Vilhelmina Berg)", female, 27,0,1, 14742, 31.3333, S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)", female, 14,1,0, 237736, 30.0708, C

Note: Refer to the visible test case for better reference.

Sample Test Cases

BarPlotOf...

1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15 survival_counts = data['Survived'].value_counts()
16 survival_counts.plot(kind='bar')
17 plt.title('Survival Count')
18 plt.xlabel('Survived')
19 plt.ylabel('Count')
20 plt.show()

Average time: 0.502 s
Maximum time: 0.502 s
1 out of 1 shown test case(s) passed

Test case 1
Expected output: Survival Count bar chart showing counts for Survived (0, 1).
Actual output: Survival Count bar chart showing counts for Survived (0, 1).

Terminal

Test Cases

Prev

Next

View recent browsing across windows and devices

Course

New Tab

GitHub Dashboard

View 1 - @penwainmed12's u X

New Tab

FPS 79

milatac.codetantra.com/secure/course.jsp?eudd=696c54060aba96214c8b7d5e4/contents/696c54060aba96214c8b7d5d/67e2ba397cd0b912d660e689c

00%

Support Logout

DE TANTRA

Home Learn Anywhere

202301090044@milatac.ac.in

5.2.2. Histogram of passenger information of Titanic

Write a Python code to plot a histogram for the distribution of the 'Age' column from the Titanic dataset. The histogram should display the frequency of different age ranges with the following specifications:
1. Use 30 bins for the histogram
2. Set the edge color of the bars to black (k).
3. Label the x-axis as 'Age' and the y-axis as 'Frequency'.
4. Add the title "Age Distribution" to the histogram.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Branson, Mr. Owen Harris", male, 21,1,0,0, 21171,7.25,S
2,1,3,"Cunha, Mrs. John Bradley (Florence Briggs Thayer)", female, 16,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina", female, 26,0,0,STON/O2, 3101282,7.92,S
4,1,1,"Pettersen, Mrs. Jacobus Math (Lily May Peel)", female, 25,1,0,312882,53.1,C223,S
5,0,3,"Allen, Mr. William Henry", male, 35,0,0,873498,8.65,S
6,0,3,"Moran, Mr. James", male, 14,0,1,10677,9.58,S
7,0,1,"McCarthy, Mr. Timothy J", male, 54,0,0,17463,51.6625,C40,S
8,0,3,"Palsson, Master. Gosta Leonard", male, 2,3,1,349909,11.875,S
9,1,3,"Jansson, Mrs. Oscar W (Ellisabeth Vilhelmina Berg)", female, 27,0,1,14742,11.1333,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achen)", female, 14,1,0,237736,30.4708,C

Note: Refer to the visible test case for better reference.

Sample Test Cases

Histogram...

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # Data Cleaning
8 data['Age'].fillna(data['Age'].median(), inplace=True)
9 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
11
12 # Convert categorical features to numeric
13 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
14 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
15 plt.hist(data['Age'], bins=30, edgecolor='k')
16 plt.title('Age Distribution')
17 plt.xlabel('Age')
18 plt.ylabel('Frequency')
19 plt.show()
```

Average time: 0.474 s, 474.00 ms; Maximum time: 0.474 s, 474.00 ms; 1 out of 1 shown test case(s) passed

Test case 1: Expected output (Age Distribution histogram) vs Actual output (Age Distribution histogram)

Terminal Test Cases

Prev Abort Submit Next

Course

Course

New Tab

GitHub Dashboard

View 1 - @pewangalmed12's u

New Tab

FPS 96

mitaoe.codetantra.com/secure/course.jpg?eucdd=696c54060aba96214c8b7d5e4/contents/696c54060aba96214c8b7d5e4/696c54060aba96214c8b7d5e4/677cdd452372a2277b32ab1c

CODETANTRA

202301090044@mitaoe.ac.in

Logout

5.2.1. Titanic Dataset

Write a Python program to analyze and visualize data from the Titanic dataset based on the following instructions:

Dataset Information:

The dataset is stored in a CSV file named `titanic.csv` and has been loaded using the `pandas` library. It contains the following columns:

- `Pclass`: Passenger class (1 = First, 2 = Second, 3 = Third).
- `Gender`: Gender of the passenger (male/female).
- `Age`: Age of the passenger.
- `Survived`: Survival status (0 = Did not survive, 1 = Survived).
- `Fare`: Ticket fare paid by the passenger.

Visualization:

To represent these trends, you will create 5 visualizations using `Matplotlib`. The visualizations should be arranged in a 3x2 grid (3 rows and 2 columns).

Visualization Details:

Write the code to create a series of visualizations as follows:

Bar Plot (Pclass Distribution):

- Create a bar plot to show the distribution of passengers across the different passenger classes (`Pclass`).
- Use the color `skyblue` for the bars.
- Title the plot as "Passenger Class Distribution".
- Label the x-axis as "Pclass" and the y-axis as "Count".

Pie Chart (Gender Distribution):

- Create a pie chart to display the distribution of male and female passengers.
- Use `lightblue` for males and `lightcoral` for females.
- Include percentages on the slices (use `autopct='%1.1f%%'`).
- Title the plot as "Gender Distribution".

Histogram (Age Distribution):

- Create a histogram to visualize the distribution of passengers' ages.
- Use `lightgreen` for the bars with black edges (`edgecolor = 'black'`).
- Set the number of bins to 8 for the histogram.
- Title the plot as "Age Distribution".

Bar Plot (Survival Count):

- Label the x-axis as "Age" and the y-axis as "Frequency".
- Create a bar plot to show the count of passengers who survived and those who did not, based on the `Survived` column.
- Use the colors `lightblue` for survivors (1) and `lightcoral` for non-survivors (0).
- Title the plot as "Survival Count".
- Label the x-axis as "Survived (0 = No, 1 = Yes)" and the y-axis as "Count".

Scatter Plot (Fare vs Age):

- Create a scatter plot to visualize the relationship between the `Fare` and `Age` of passengers.
- Use orange for the data points.
- Title the plot as "Fare vs Age".
- Label the x-axis as "Age" and the y-axis as "Fare".

Note: Refer to the displayed plot in the sample test cases for better understanding.

titanicDat...

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

526

527

528

529

530

531

532

533

534

535

536

537

538

539

540

541

542

543

544

545

546

547

548

549

550

551

552

553

554

555

556

557

558

559

560

561

562

563

564

565

566

567

568

569

570

571

572

573

574

575

576

577

578

579

580

581

582

583

584

585

586

587

588

589

590

591

592

593

594

595

596

597

598

599

600

601

602

603

604

605

606

607

608

609

610

611

612

613

614

615

616

617

618

619

620

621

622

623

624

625

626

627

628

629

630

631

632

633

634

635

636

637

638

639

640

641

642

643

644

645

646

647

648

649

650

651

652

653

654

655

656

657

658

659

660

661

662

663

664

665

666

667

668

669

670

671

672

673

674

675

676

677

678

679

680

681

682

683

684

685

686

687

688

689

690

691

692

693

694

695

696

697

698

699

700

701

702

703

704

705

706

707

708

709

710

711

712

713

714

715

716

717

718

719

720

721

722

723

724

725

726

727

728

729

730

731

732

733

734

735

736

737

738

739

740

741

742

743

744

745

746

747

748

749

750

751

752

753

754

755

756

757

758

759

760

761

762

763

764

765

766

767

768

769

770

771

772

773

774

775

776

777

778

779

780

781

782

783

784

785

786

787

788

789

790

791

792

793

794

795

796

797

798

799

800

801

802

803

804

805

806

807

808

809

810

811

812

813

814

815

816

817

818

819

820

821

822

823

824

825

826

827

828

829

830

831

832

833

834

835

836

837

838

839

840

841

842

843

844

845

846

847

848

849

850

851

852

853

854

855

856

857

858

859

860

861

862

863

864

865

866

867

868

869

870

871

872

873

874

875

876

877

878

879

880

881

882

883

884

885

886

887

888

889

890

891

892

893

894

895

896

897

898

899

900

901

902

903

904

905

906

907

908

909

910

911

912

913

914

915

916

917

918

919

920

921

922

923

924

925

926

927

928

929

930

931

932

933

934

935

936

937

938

939

940

941

942

943

944

945

946

947

948

949

950

951

952

953

954

955

956

957

958

959

960

961

962

963

964

965

966

967

968

969

970

971

972

973

974

975

976

977

978

979

980

981

982

983

984

985

986

987

988

989

990

991

992

993

994

995

996

997

998

999

1000

Test case 1

Expected output

Actual output

Terminal

Test cases

Previous

Next

Course

Course

New Tab

GitHub Dashboard

View 1 - @perwainmad123 u X

New Tab

FPS 65

initiaee.codedtantra.com/secure/course.jpg?eudd=696c54060aba96214c8b7d5e4/contents/696c54060aba96214c8b7d5e4/696c54060aba96214c8b7d5e4/675a9d2c09f56d22faa9dc0

00%

Support Logout

CODETANTRA

Home Learn Anywhere

202501090044@initiaee.ac.in

5.1.1. Stacked Plot

0.438 s

438.00 ms

0.438 s

438.00 ms

1 out of 1 shown test case(s) passed

Test case 1

Expected output

Actual output

Terminal

Test Cases

Prev

Next

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

526

527

528

529

530

531

532

533

534

535

536

537

538

539

540

541

542

543

544

545

546

547

548

549

550

551

552

553

554

555

556

557

558

559

560

561

562

563

564

565

566

567

568

569

570

571

572

573

574

575

576

577

578

579

580

581

582

583

584

585

586

587

588

589

590

591

592

593

594

595

596

597

598

599

600

601

602

603

604

605

606

607

608

609

610

611

612

613

614

615

616

617

618

619

620

621

622

623

624

625

626

627

628

629

630

631

632

633

634

635

636

637

638

639

640

641

642

643

644

645

646

647

648

649

650

651

652

653

654

655

656

657

658

659

660

661

662

663

664

665

666

667

668

669

670

671

672

673

674

675

676

677

678

679

680

681

682

683

684

685

686

687

688

689

690

691

692

693

694

695

696

697

698

699

700

701

702

703

704

705

706

707

708

709

710

711

712

713

714

715

716

717

718

719

720

721

722

723

724

725

726

727

728

729

730

731

732

733

734

735

736

737

738

739

740

741

742

743

744

745

746

747

748

749

750

751

752

753

754

755

756

757

758

759

760

761

762

763

764

765

766

767

768

769

770

771

772

773

774

775

776

777

778

779

780

781

782

783

784

785

786

787

788

789

790

791

792

793

794

795

796

797

798

799

800

801

802

803

804

805

806

807

808

809

810

811

812

813

814

815

816

817

818

819

820

821

822

823

824

825

826

827

828

829

830

831

832

833

834

835

836

837

838

839

840

841

842

843

844

845

846

847

848

849

850

851

852

853

854

855

856

857

858

859

860

861

862

863

864

865

866

867

868

869

870

871

872

873

874

875

876

877

878

879

880

881

882

883

884

885

886

887

888

889

890

891

892

893

894

895

896

897

898

899

900

901

902

903

904

905

906

907

908

909

910

911

912

913

914

915

916

917

918

919

920

921

922

923

924

925

926

927

928

929

930

931

932

933

934

935

936

937

938

939

940

941

942

943

944

945

946

947

948

949

950

951

952

953

954

955

956

957

958

959

960

961

962

963

964

965

966

967

968

969

970

971

972

973

974

975

976

977

978

979

980

981

982

983

984

985

986

987

988

989

990

991

992

993

994

995

996

997

998

999

1000

1001

1002

1003

1004

1005

1006

1007

1008

1009

1010

1011

1012

1013

1014

1015

1016

1017

1018

1019

1020

1021

1022

1023

1024

1025

1026

1027

1028

1029

1030

1031

1032

1033

1034

1035

1036

1037

1038

1039

1040

1041

1042

1043

1044

1045

1046

1047

1048

1049

1050

1051

1052

1053

1054

1055

1056

1057

1058

1059

1060

1061

1062

1063

1064

1065

1066

1067

1068

1069

1070

1071

1072

1073

1074

1075

1076

1077

1078

1079

1080

1081

1082

1083

1084

1085

1086

1087

1088

1089

1090

1091

1092

1093

1094

1095

1096

1097

1098

1099

1100

1101

1102

1103

1104

1105

1106

1107

1108

1109

1110

1111

1112

1113

1114

1115

1116

1117

1118

1119

1120

1121

1122

1123

1124

1125

1126

1127

1128

1129

1130

1131

1132

1133

1134

1135

1136

1137

1138

1139

1140

1141

1142

1143

1144

1145

1146

1147

1148

1149

1150

1151

1152

1153

1154

1155

1156

1157

1158

1159

1160

1161

1162

1163

1164

1165

1166

1167

1168

1169

1170

1171

1172

1173

1174

1175

1176

1177

1178

1179

1180

1181

1182

1183

1184

1185

1186

1187

1188

1189

1190

1191

1192

1193

1194

1195

1196

1197

1198

1199

1200

1201

1202

1203

1204

1205

1206

1207

1208

1209

1210

1211

1212

1213

1214

1215

1216

1217

1218

1219

1220

1221

1222

1223

1224

1225

1226

1227

1228

1229

1230

1231

1232

1233

1234

1235

1236

1237

1238

1239

1240

1241

1242

1243

1244

1245

1246

1247

1248

1249

1250

1251

1252

1253

1254

1255

1256

1257

1258

1259

1260

1261

1262

1263

1264

1265

1266

1267

1268

1269

1270

1271

1272

1273

1274

1275

1276

1277

1278

1279

1280

1281

1282

1283

1284

1285

1286

1287

1288

1289

1290

1291

1292

1293

1294

1295

1296

1297

1298

1299

1300

1301

1302

1303

1304

1305

1306

1307

1308

1309

1310

1311

1312

1313

1314

1315

1316

1317

1318

1319

1320

1321

1322

1323

1324

1325

1326

1327

1328

1329

1330

1331

1332

1333

1334

1335

1336

1337

1338

1339

1340

1341

1342

1343

1344

1345

1346

1347

1348

1349

1350

1351

1352

1353

1354

1355

1356

1357

1358

1359

1360

1361

1362

1363

1364

1365

1366

1367

1368

1369

1370

1371

1372

1373

1374

1375

1376

1377

1378

1379

1380

1381

1382

1383

1384

1385

1386

1387

1388

1389

1390

1391

1392

1393

1394

1395

1396

1397

1398

1399

1400

1401

1402

1403

1404

1405

1406

1407

1408

1409

1410

1411

1412

1413

1414

1415

1416

1417

1418

1419

1420

1421

1422

1423

1424

1425

1426

1427

1428

1429

1430

1431

1432

1433

1434

1435

1436

1437

1438

1439

1440

1441

1442

1443

1444

1445

1446

1447

1448

1449

1450

1451

1452

1453

1454

14

View recent browsing across windows and devices

Course

New Tab

GitHub Dashboard

View I - @penwagained12's u X

New Tab

FPS 112

mitaoe.codedtantra.com/secure/course.jpg?eudd=696c54060aba96214c8b7d5e4/contents/696c54060aba96214c8b7d5e4/696c54060aba96214c8b7d5e4/67e3905d7028471d94dae951

00%

Support Logout

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

100%

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.

2. Calculate the survival rate by embarked location (Embarked_S).

3. Calculate the survival rate by family size (FamilySize).

4. Calculate the survival rate by being alone (IsAlone).

5. Get the average fare by passenger class (Pclass).

6. Get the average age by passenger class (Pclass).

7. Get the average age by survival status (Survived).

8. Get the average fare by survival status (Survived).

9. Get the number of survivors by class (Pclass).

10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Owen Harris	male	22	1	0	A/5 21171	7.25	0	0
2	1	1	Mr. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17569	71.2833	C85	C
3	1	3	Mr. Louis Green	male	22	0	0	STON/O2	53.1	0	0
4	1	1	Mr. William Henry	male	31	0	0	113803	53.1	0	0
5	0	3	Mr. James	male	26	0	0	5136	51.0	0	0
6	0	3	Mr. James	male	26	0	0	5136	51.0	0	0
7	0	3	Mr. James	male	26	0	0	5136	51.0	0	0
8	0	3	Mr. James	male	26	0	0	5136	51.0	0	0
9	0	3	Mr. James	male	26	0	0	5136	51.0	0	0
10	0	3	Mr. James	male	26	0	0	5136	51.0	0	0

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked

1, 0, 3, "Mr. Owen Harris", male, 22, 1, 0, "A/5 21171", 7.25, 0, 0

2, 1, 1, "Mr. John Bradley (Florence Briggs Thayer)", female, 38, 1, 0, "PC 17569", 71.2833, "C85", "C"

3, 1, 3, "Mr. Louis Green", male, 22, 0, 0, "STON/O2", 53.1, 0, 0

4, 1, 1, "Mr. William Henry", male, 31, 0, 0, "113803", 53.1, 0, 0

5, 0, 3, "Mr. James", male, 26, 0, 0, "5136", 51.0, 0, 0

6, 0, 3, "Mr. James", male, 26, 0, 0, "5136", 51.0, 0, 0

7, 0, 3, "Mr. James", male, 26, 0, 0, "5136", 51.0, 0, 0

8, 0, 3, "Mr. James", male, 26, 0, 0, "5136", 51.0, 0, 0

9, 0, 3, "Mr. James", male, 26, 0, 0, "5136", 51.0, 0, 0

10, 0, 3, "Mr. James", male, 26, 0, 0, "5136", 51.0, 0, 0

Note: Refer to the visible test case for better reference.

Sample Test Cases

1

Test case 1

Expected output

Actual output

1: 0.521638

1: 0.521638

2: 0.472826

2: 0.472826

3: 0.242393

3: 0.242393

Name: Survived, dtype: float64

Name: Survived, dtype: float64

Embarked_S

Embarked_S

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

526

527

528

529

530

531

532

533

534

535

536

537

538

539

540

541

542

543

544

545

546

547

548

549

550

551

552

553

554

555

556

557

558

559

560

561

562

563

564

565

566

567

568

569

570

571

572

573

574

575

576

577

578

579

580

581

582

583

584

585

586

587

588

589

590

591

592

593

594

595

596

597

598

599

600

601

602

603

604

605

606

607

608

609

610

611

612

613

614

615

616

617

618

619

620

621

622

623

624

625

626

627

628

629

630

631

632

633

634

635

636

637

638

639

640

641

642

643

644

645

646

647

648

649

650

651

652

653

654

655

656

657

658

659

660

661

662

663

664

665

666

667

668

669

670

671

672

673

674

675

676

677

678

679

680

681

682

683

684

685

686

687

688

689

690

691

692

693

694

695

696

697

698

699

700

701

702

703

704

705

706

707

708

709

710

711

712

713

714

715

716

717

718

719

720

721

722

723

724

725

726

727

728

729

730

731

732

733

734

735

736

737

738

739

740

741

742

743

744

745

746

747

748

749

750

751

752

753

754

755

756

757

758

759

760

761

762

763

764

765

766

767

768

769

770

771

772

773

774

775

776

777

778

779

780

781

782

783

784

785

786

787

788

789

790

791

792

793

794

795

796

797

798

799

800

801

802

803

804

805

806

807

808

809

810

811

812

813

814

815

816

817

818

819

820

821

822

823

824

825

826

827

828

829

830

831

832

833

834

835

836

837

838

839

840

841

842

843

844

845

846

847

848

849

850

851

852

853

854

855

856

857

858

859

860

861

862

863

864

865

866

867

868

869

870

871

872

873

874

875

876

877

878

879

880

881

882

883

884

885

886

887

888

889

890

891

892

893

894

895

896

897

898

899

900

901

902

903

904

905

906

907

908

909

910

911

912

913

914

915

916

917

918

919

920

921

922

923

924

925

926

927

928

929

930

931

932

933

934

935

936

937

938

939

940

941

942

943

944

945

946

947

948

949

950

951

952

953

954

955

956

957

958

959

960

961

962

963

964

965

966

967

968

969

970

971

972

973

974

975

976

977

978

979

980

981

982

983

984

985

986

987

988

989

990

991

992

993

994

995

996

997

998

999

1000

Course

Course

New Tab

GitHub Dashboard

View | @pawangalmed12's u X

New Tab

FPS 118

mitaoe.codedtantra.com/secure/course.jpg?eudd=696c54060aba96214c8b7d5e7/contents/696c54060aba96214c8b7d5e7/67e378a17028471d94dad0a6

00%

202501090044@mitaoe.ac.in Support Logout

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

100%

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

- Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
- Convert the 'Sex' column to numeric values (male: 0, female: 1).
- One-hot encode the 'Embarked' column, dropping the first category.
- Get the mean age of passengers.
- Get the median fare of passengers.
- Get the number of passengers by class.
- Get the number of passengers by gender.
- Get the number of passengers by survival status.
- Calculate the survival rate of passengers.
- Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1, 0, 3, "Braund, Mr. Owen Harris", male, 22, 1, 0, A/5 21371, 7.25, 5
2, 1, 1, "Cumings, Mrs. John Bradley (Florence Briggs Thayer)", female, 38, 1, 0, PC 17599, 71.3533, C85, C
3, 1, 3, "Heikkinen, Miss. Laina", female, 26, 0, 0, STON/O2, 31.0002, 7.935, 3
4, 1, 1, "Allenby, Mrs. Thomas (Elizabeth Mary Hall)", female, 25, 1, 0, 114841, 51.1, C123, S
5, 0, 2, "Adler, Mr. William Harry", male, 35, 0, 0, 373458, 8.45, 2
6, 0, 3, "Mann, Mr. James", male, 8, 0, 39877, 8.4583, Q
7, 0, 1, "McCarthy, Mr. Timothy J", male, 54, 0, 0, 17463, 51.8625, C64, S
8, 0, 3, "Palsson, Master. Gosta Leonard", male, 2, 1, 1, 349909, 16.495, 3
9, 1, 3, "Johnson, Mrs. Oscar W (Ellenesth Vilhelmina Berg)", female, 37, 0, 1, 547742, 13.3358, S
10, 1, 1, "Nasser, Mrs. Nicholas (Adele Achem)", female, 54, 1, 0, 297276, 56.8786, C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

titanicDat...

100%

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7
8 # 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
9 data['IsAlone'] = np.where(data['FamilySize'] == 0, 1, 0)
10
11 # 2. Convert 'Sex' to numeric (male: 0, female: 1)
12 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
13
14 # 3. One-hot encode the 'Embarked' column
15 data = pd.get_dummies(data, columns=['Embarked'])
16
17 # 4. Get the mean age of passengers
18 mean_age = data['Age'].mean()
19 print(mean_age)
20
21 # 5. Get the median fare of passengers
22 median_fare = data['fare'].median()
23 print(median_fare)
24
25 # 6. Get the number of passengers by class
26 print(data['Pclass'].value_counts())
27
28 # 7. Get the number of passengers by gender
29 print(data['Sex'].value_counts())
30
31 # 8. Get the number of passengers by survival status
32 print(data['Survived'].value_counts())
33
34 # 9. Calculate the overall survival rate
```

Average time: 0.050 s, 10.00 ms

Maximum time: 0.050 s, 10.00 ms

1 out of 1 shown test case(s) passed

Test case 1

Expected output

Actual output

29.69911764705882

29.69911764705882

16.4543

16.4543

3 - 401

3 - 401

1 - 226

1 - 226

2 - 184

2 - 184

Name: Pclass, dtype: int64

Name: Pclass, dtype: int64

Terminal

Test Cases

Prev

Reset

Submit

Next

View recent browsing across windows and devices

Course

New Tab

GitHub Dashboard

View 1 - @pawangalrmed123 u X

New Tab

FPS 126

mitaoe.codedtantra.com/secure/course.jpg?eudd=696c54060aba96214c8b7d5e4/contents/696c54060aba96214c8b7d5e4/67e2b9309025661d/3aa574f

00%

202301090044@mitaoe.ac.in Support Logout

CODETANTRA

Home Learn Anywhere

4.2.5. Titanic Dataset Analysis and Data Cleaning

3/29

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

- Display the first 5 rows of the dataset
- Display the last 5 rows of the dataset
- Get the shape of the dataset (number of rows and columns).
- Get a summary of the dataset (using `.info()`).
- Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
- Check for missing values and display the count of missing values for each column.
- Fill missing values in the 'Age' column with the median age.
- Fill missing values in the 'Embarked' column with the most frequent value (mode).
- Drop the 'Cabin' column due to many missing values.
- Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,0,7171,7.25,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17509,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2, 3181282,7.925,S
4,1,1,"Patricello, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Kallies, Mr. William Henry",male,35,0,0,17460,8.69,S
6,0,3,"Moran, Mr. James",male,30,0,108677,8.683,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,C46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,342909,21.075,S
9,1,3,"Tomecek, Mrs. Janca W (Jilkaetha Vilhelmina Berg)",female,27,0,1,147242,11.5319,S
10,1,4,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,217706,38.0788,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

titanicDat...

3/29

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # 1. Display the first 5 rows of the dataset
8 print(data.head())
9
10 # 2. Display the last 5 rows of the dataset
11 print(data.tail())
12
13 # 3. Get the shape of the dataset (number of rows and columns)
14 print(data.shape)
15
16 # 4. Get a summary of the dataset
17 print(data.info())
18
19 # 5. Get basic statistics of the dataset
20 print(data.describe())
21
22 # 6. Check for missing values and display the count
23 print(data.isnull().sum())
24
25 # 7. Fill missing values in the 'Age' column with the median age
26 data['Age'].fillna(data['Age'].median(), inplace=True)
27
28 # 8. Fill missing values in the 'Embarked' column with the most frequent value (mode)
29 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
30
31 # 9. Drop the 'Cabin' column due to many missing values
32 data.drop(columns='Cabin', inplace=True)
```

Average time: 0.160 s, 100.00 ms

Maximum time: 0.160 s, 100.00 ms

1 out of 1 shown test case(s) passed

Test case 1

Debug

Expected output

Actual output

---PassengerId---Survived---Pclass---Name---Sex---Age---SibSp---Parch---Ticket---Fare---Cabin---Embarked---
8-----1-----0-----3-----Braund, Mr. Owen Harris-----male-----22-----1-----0-----PC 17509-----71.2833-----C85-----S
1-----1-----2-----1-----Cumings, Mrs. John Bradley (Florence Briggs Thayer)-----female-----38-----1-----0-----PC 17509-----71.2833-----C85-----S
2-----1-----3-----1-----Heikkinen, Miss. Laina-----female-----26-----0-----0-----STON/O2, 3181282-----7.925-----S
3-----1-----1-----1-----Patricello, Mrs. Jacques Heath (Lily May Peel)-----female-----35-----1-----0-----113803-----53.1-----C123-----S
4-----0-----3-----1-----Kallies, Mr. William Henry-----male-----35-----0-----0-----17460-----8.69-----S
5-----0-----3-----1-----Moran, Mr. James-----male-----30-----0-----108677-----8.683-----Q
6-----0-----1-----1-----McCarthy, Mr. Timothy J-----male-----54-----0-----0-----17463-----51.8625-----C46-----S
7-----0-----3-----1-----Palsson, Master. Gosta Leonard-----male-----2-----3-----1-----342909-----21.075-----S
8-----1-----3-----1-----Tomecek, Mrs. Janca W (Jilkaetha Vilhelmina Berg)-----female-----27-----0-----1-----147242-----11.5319-----S
9-----1-----4-----1-----Nasser, Mrs. Nicholas (Adele Achem)-----female-----14-----1-----0-----217706-----38.0788-----C

Terminal

Test cases

Prev

Reset

Submit

Next

Course

Course

New Tab

GitHub Dashboard

View 1 - @penwagimad12's u

New Tab

FPS 74

mitaoe.codetantra.com/secure/course.jpg?eucdd=696c54060aba96214c8b7d5e7/contents/696c54060aba96214c8b7d5e7/6788eeq89f79b6ea9e3ee1

CODETANTRA Home Learn Anywhere

202501090044@mitaoe.ac.in Support Logout

4.2.A. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the following columns: Date, Product, Quantity, Price, and City

For each date, find all pairs of products that were sold together (i.e., two products sold on the same date)

Output the product pairs that was sold most frequently

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	10	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	6	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	5	20	Chicago
2025-01-05	Product C	3	10	Los Angeles

Explanation:

Transactions:

2025-01-01: Product A, Product B

2025-01-02: Product A, Product C

2025-01-03: Product B, Product A

2025-01-04: Product C, Product B

2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

Product A and Product B: Appear in transactions on 2025-01-01 and 2025-01-03

Product A and Product C: Appear in transactions on 2025-01-02 and 2025-01-05

Product B and Product C: Appear in transactions on 2025-01-04

Most Frequent Product Combinations:

Product A and Product B (2 times)

Product A and Product C (2 times)

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

frequent...

1 import pandas as pd

2 from itertools import combinations

3 from collections import Counter

4

5 # Prompt user to input the file name

6 file_name = input()

7

8 # Read data from the specified CSV file

9 df = pd.read_csv(file_name)

10 date_products = []

11

12 for date, group in df.groupby('Date'):

13 products = group['Product'].unique()

14 if len(products) > 1:

15 date_products.append(products)

16

17 # Count product pairs

18 pair_counter = Counter()

19

20 for products in date_products.values():

21 # Sort to avoid duplicate pairs like (A, B) and (B, A)

22 pairs = combinations(sorted(products), 2)

23 pair_counter.update(pairs)

24

25 # Find the maximum frequency

26 if pair_counter:

27 max_count = max(pair_counter.values())

28

29 # Output the most frequent product pairs

30 for pair, count in pair_counter.items():

31 if count == max_count:

32 print(f'{pair[0]} and {pair[1]}: {count} times')

33 else:

34 print("No product combinations found.")

Average time

0.025 s

24.67 ms

Maximum time

0.026 s

26.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

Expected output

Actual output

Product A and Product B: 2 times

Product A and Product C: 2 times

Product A and Product C: 2 times

Terminal

Test Cases

Prev

Next

Course

Course

New Tab

GitHub Dashboard

View 1 - @pawargalxvad72su

New Tab

HPS 72

mitaoe.codetantra.com/secure/course.jsp?eucld=698c54860aba96214c8b7d5e#contents/698c54860aba96214c8b7d5e/698c54860aba96214c8b7d5e/67861553a50fe56b8dc9a6e

CODETANTRA

Home

Learn Anywhere

20210100044@mitaoe.ac.in

Support

Logout

4.2.3. City That Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	18	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	18	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	18	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 # write the code..
10 city_sales = df.groupby("City")["Quantity"].sum()
11 best_city = city_sales.idxmax()
12 # Display the result
13 print(f"City sold the most products: {best_city}")
14
```

Average time: 0.018 s
18.33 ms

Maximum time: 0.020 s
20.00 ms

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 48 ms

Expected output: sales_data.csv
City sold the most products: Los Angeles

Actual output: sales_data.csv
City sold the most products: Los Angeles

Terminal Test cases

Prev

Next

Submit

Next

Course

Course

New Tab

GitHub Dashboard

View 1 - @penwainmed123 u X

New Tab

FPS 16

mitaoe.codetantra.com/secure/course.jpg?eucdd=696c54060aba96214c8b7d5e7/contents/696c54060aba96214c8b7d5e7/67861370a50fe96b8dcf9828

CODETANTRA Home Learn Anywhere

202501090044@mitaoe.ac.in Support Logout

4.2.2. Best Selling Product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date	Product	Quantity	Price	City
2023-01-01	Product A	25	10	New York
2023-01-05	Product B	15	12	Los Angeles
2023-01-08	Product A	7	20	New York
2023-01-02	Product C	30	8	Chicago
2023-01-03	Product B	15	15	Chicago
2023-01-04	Product A	6	20	Los Angeles
2023-01-04	Product C	6	20	New York
2023-01-04	Product B	15	10	Los Angeles
2023-01-06	Product A	15	10	Chicago
2023-01-09	Product C	10	10	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 product_sales = df.groupby("Product")["Quantity"].sum()
10 # Find the product with the highest total quantity sold
11 # Use group by 'product' and sum the 'Quantity', then find the index (product name)
12 # And the max value.
13 product_sales = df.groupby('Product')['Quantity'].sum()
14 best_product = product_sales.idxmax()
15 highest_quantity = product_sales.max()
16
17 # Display the result
18 print(f"Best selling product: {best_product}")
19 print(f"Total quantity sold: {highest_quantity}")
20
```

Average time: 0.024 s, 24.00 ms

Maximum time: 0.025 s, 25.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test Case 1

Expected output:
Best selling product: Product A
Total quantity sold: 23

Actual output:
Best selling product: Product A
Total quantity sold: 23

Terminal Test Cases

Prev Restart Submit Next



View recent browsing across windows and devices

Course

New Tab

GitHub Dashboard

View 1 · @pewangalmed123 u X

FPS 0

mitaoe.codetantra.com/secure/course.jpg?eucdd=696c54060aba96214c8b7d5e7/contents/696c54060aba96214c8b7d5e7/67360994a50fe96b8dc6747

00%

Support Logout

4.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

Date	Product	Quantity	Price	City
2023-01-01	Product A	5	20	New York
2023-01-01	Product B	3	15	Los Angeles
2023-01-02	Product A	7	20	New York
2023-01-02	Product C	4	30	Chicago
2023-01-03	Product B	2	15	Chicago
2023-01-03	Product A	6	20	Los Angeles
2023-01-04	Product C	5	30	New York
2023-01-04	Product B	5	15	Los Angeles
2023-01-05	Product A	5	20	Chicago
2023-01-05	Product C	3	30	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8 df['Date'] = pd.to_datetime(df['Date'])
9 df['Month'] = df['Date'].dt.strftime('%Y-%m')
10 df['Total Sales'] = df['Quantity'] * df['Price']
11
12
13 # Find the month with the highest total sales
14 sales_by_month = df.groupby('Month')['Total Sales'].sum()
15 best_month = sales_by_month.idxmax()
16
17 highest_sales = sales_by_month.max()
18
19 print(f"Best month: {best_month}")
20 print(f"Total sales: ${highest_sales:.2f}")
21
```

Average time: 0.025 s, 20.00 ms

Maximum time: 0.032 s, 32.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test Case 1

Expected output:
sales_data.csv
Best month: 2023-01
Total sales: \$1230.00

Actual output:
sales_data.csv
Best month: 2023-01
Total sales: \$1230.00

Terminal Test Cases

Prev Next Submit

Course

Course

New Tab

GitHub Dashboard

View 1 - @pawangalmed123 u X

FPS 16

mitaoe.codetantra.com/secure/course.jpg?eucdd=698c54060aba96214c8b7d5e4/contents/698c54060aba96214c8b7d5e4/698c54060aba96214c8b7d5e4/667d60f2305ba61af5a2ce7

00%

202501090044@mitaoe.ac.in Support Logout

4.5.3. Student Information

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:
Refer to the displayed test cases for better understanding.

Sample Test Cases

studentin...

```
1 import pandas as pd
2
3 # Read the text file into a DataFrame
4 file = input()
5 data = pd.read_csv(file, sep=";", header=None, names=["Name", "Age", "Grade"])
6 print("First five rows:")
7 print(data.head())
8 avround(data['Age'].mean(),2)
9 print("Average age:",av)
10 fd=data[data['Grade'].isin(['A','B'])]
11 print("Students with a grade up to B")
12 print(fd)
```

Average time: 0.061 s
61.00 ms

Maximum time: 0.073 s
73.00 ms

1 out of 1 shown test case(s) passed
1 out of 1 hidden test case(s) passed

Test Case 1

Expected output: studentinfo.txt
First five rows:
Name Age Grade
0 John 25 B
1 Alice 22 B
2 Emma 24 A

Actual output: studentinfo.txt
First five rows:
Name Age Grade
0 John 25 B
1 Alice 22 B
2 Emma 24 A

Terminal Test Cases

Prev

Reset

Submit

Next

Course

Course

New Tab

GitHub Dashboard

View 1 · @pewangalmed12's u · X

FPS 84

initaoe.codetantra.com/secure/course.jpg?eudd=696c54060aba96214c8b7d5e4/contents/696c54060aba96214c8b7d5e4/667aa30728220463f649bbb62

CODETANTRA

Home Learn Anywhere

202301090044@initaoe.ac.in Support Logout

4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column Gender with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the Age column.
- Display the DataFrame after deleting the column.

Sample Test Cases

datafram...

```
1 import pandas as pd
2
3 # Provided dictionary of lists
4 data = {
5     'Name': ['Alice', 'Bob', 'Charlie'],
6     'Age': [25, 30, 35]
7 }
8
9 # Convert the dictionary to a DataFrame
10 df = pd.DataFrame(data)
11
12 # Display the original DataFrame
13 print("Original DataFrame:")
14 print(df)
15
16 # Adding a new row
17 name = input("Enter name for new row: ")
18 age = int(input("Enter age for new row: "))
19
20 new_row = {'Name': name, 'Age': age}
21 df.loc[len(df)] = new_row
22
23 print("\nAfter adding a row:")
24 print(df)
25
26 # Display the DataFrame after adding a new row
27 print("After adding a row:\n",df)
28
29
30 # Modifying a row
31 row_index = int(input("\nEnter row index to modify age: "))
32 new_age = int(input("Enter new age: "))
33
34 df.at[row_index, 'Age'] = new_age
```

Average time: 0.203 s
203.00 ms

Maximum time: 0.217 s
217.00 ms

0 out of 1 shown test case(s) passed
0 out of 1 hidden test case(s) passed

Test case 1

Debug

Output mismatch

Expected output

Original dataframe:

None Age

0 Alice 25

1 Bob 30

2 Charlie 35

Actual output

Original dataframe:

None Age

0 Alice 25

1 Bob 30

2 Charlie 35

Terminal

Test cases

Prev

Next

Submit

Course

Course

New Tab

GitHub Dashboard

View 1 - @pewangalmed12's u

FPS 8

initaoe.codetantra.com/secure/course.jpg?eudd=696c54060aba96214c8b7d5e4/contents/696c54060aba96214c8b7d58/696c54060aba96214c8b7d59/667a7e0426220463f6497a3c

CODETANTRA Home Learn Anywhere

202301090044@mitase.ac.in Support Logout

4.5.1. Pandas - series creation and manipulation

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format

Sample Test Cases

seriesMa...

```
1 import pandas as pd
2
3 # Take inputs from the user to create a list of numbers
4 numbers = list(map(int, input().split()))
5 Series=pd.Series(numbers)
6 # Grouping by even and odd numbers and calculating the mean
7 grouped = Series.groupby(Series%2==0).mean()
8
9 # Display the mean of even and odd numbers with labels
10 grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
11 print("Mean of even and odd numbers:")
12 print(grouped)
13
```

Average time: 0.018 s10.00 ms

Maximum time: 0.022 s22.00 ms

3 out of 3 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test Case 1

Expected output

Mean of even and odd numbers:

Odd: 5.8

Even: 6.8

Output: #10064

Actual output

Mean of even and odd numbers:

Odd: 5.8

Even: 6.8

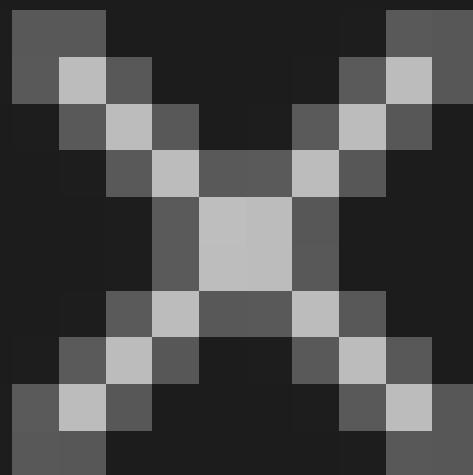
Output: #10064

Terminal

Test Cases

Prev

Next



View recent browsing across windows and devices

Course

New Tab

GitHub Dashboard

View 1 - @pewangalmed123 u X

FPS 93

initialec.codedtantra.com/secure/course.jpg?eudd=698c54060aba96214c8b7d5e9/contents/698c54060aba96214c8b7d57/67dfb4f1d1a74a4035ef4225

00%

202501090044@initialec.ac.in Support Logout

CODETANTRA

Home Learn Anywhere

1.2.7. Student Data Analysis and Operations

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- Print all student details: Display the complete details of all students, including roll numbers and marks for all subjects.
- Find total students: Determine the total number of students in the dataset.
- Print all student roll numbers: Extract and print the roll numbers of all students.
- Print Subject 1 marks: Extract and print the marks of all students in Subject 1.
- Find minimum marks in Subject 2: Identify the lowest marks in Subject 2.
- Find maximum marks in Subject 3: Identify the highest marks in Subject 3.
- Print all subject marks: Display the marks of all students for each subject.
- Find total marks of students: Compute the total marks for each student across all subjects.
- Find the average marks of each student: Compute the average marks for each student.
- Find average marks of each subject: Compute the average marks for all students in each subject.
- Find average marks of Subject 1 and Subject 2: Compute the average marks for Subject 1 and Subject 2.
- Find average marks of Subject 1 and Subject 3: Compute the average marks for Subject 1 and Subject 3.
- Find the roll number of the student with maximum marks in Subject 3: Identify the student with the highest marks in Subject 3 and print their roll number.
- Find the roll number of the student with minimum marks in Subject 2: Identify the student with the lowest marks in Subject 2 and print their roll number.
- Find the roll number of students who scored 24 marks in Subject 2: Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- Find the count of students who got less than 40 marks in Subject 1: Count the number of students who scored less than 40 marks in Subject 1.
- Find the count of students who got more than 90 marks in Subject 2: Count the number of students who scored more than 90 marks in Subject 2.
- Find the count of students who scored >= 90 in each subject: Count the number of students who scored 90 or more marks in each subject.
- Find the count of subjects in which each student scored >= 90: Determine how many subjects each student scored 90 or more marks in.
- Print Subject 1 marks in ascending order: Sort and print the marks of students in Subject 1 in ascending order.
- Print students who scored between 50 and 90 in Subject 1: Display students who scored marks between 50 and 90 in Subject 1.
- Find index positions of students who scored 79 in Subject 1: Identify the index positions of students who scored exactly 79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

Sample Test Cases

Operation...

```
1 import numpy as np
2
3 a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)
4
5 # 1. Print all student details
6 print("All student Details:\n", a)
7
8 # 2. print total students
9
10 print("Total Students:", len(a))
11
12 # 3. Print all student Roll numbers
13 print("All Student Roll Nos", a[:, 0])
14
15 # 4. Print subject 1 marks
16 print("Subject 1 Marks", a[:, 1])
17
18 # 5. print minimum marks of Subject 2
19 print("Min marks in Subject 2", np.min(a[:, 2]))
20
21 # 6. print maximum marks of Subject 3
22 print("Max marks in Subject 3", np.max(a[:, 3]))
23
24 # 7. Print All subject marks
25 print("All subject marks:", a[:, 1:])
26
27 # 8. print Total marks of students
28 print("Total Marks", np.sum(a[:, 1:], axis=1))
29
30 # 9. print average marks of each student
31 print(np.round(np.mean(a[:, 1:], axis=1), 1))
32
33
34 # 10. print average marks of each student
```

Average time

0.021 s

21.00 ms

Maximum time

0.021 s

21.00 ms

1 out of 1 shown test case(s) passed

Test case 1

Expected output

Actual output

```
[[301, 47, 77, 88.]
[302, 74, 88, 77.]
[303, 45, 56, 89.]
[304, 88, 98, 45.]
[305, 74, 88, 90.]
```

```
[[301, 47, 77, 88.]
[302, 74, 88, 77.]
[303, 45, 56, 89.]
[304, 88, 98, 45.]
[305, 74, 88, 90.]
```

Terminal

Test Cases

Prev

Next

Submit

Restart

Course

Course

New Tab

GitHub Dashboard

View 1 - @pewangalmad12's u

FPS 59

initaoe.codetantra.com/secure/course.jpg?eudd=698c54060aba96214c8b7d5e4/contents/698c54060aba96214c8b7d55/698c54060aba96214c8b7d57/6774f064f4ab787eca9da9b0

00%

Support Logout

12.6. Numpy: Searching, Sorting, Counting, Broadcasting

The given code in the editor takes a single array, array1, as space-separated integers as input from the user. Additionally, it takes the following inputs:

- search_value: The value to search for in the array.
- count_value: The value to count its occurrences in the array.
- broadcast_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

- Searching: Find the indices where search_value appears in array1 and print these indices.
- Counting: Count how many times count_value appears in array1 and print the count.
- Broadcasting: Add broadcast_value to each element of array1 using broadcasting, and print the resulting array.
- Sorting: Sort array1 in ascending order and print the sorted array.

Input Format:

- A single line containing space-separated integers representing array1.
- An integer search_value represents the value to search for in the array.
- An integer count_value represents the value to count in the array.
- An integer broadcast_value represents the value to add to each element of the array.

Output Format:

- The indices where search_value occurs in array1.
- The count of occurrences of count_value in array1.
- The array after adding the broadcast_value to each element.
- The sorted array.

Sample Test Cases

arrayOpe...

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split())))
5
6 # Searching
7 search_value = int(input("Value to search: "))
8 count_value = int(input("Value to count: "))
9 broadcast_value = int(input("Value to add: "))
10
11 # Find indices where value matches in array1
12 indices = np.where(array1 == search_value)[0]
13 # Count occurrences in array1
14 count = np.sum(array1 == count_value)
15 # Broadcasting addition
16 broadcast_res = array1 + broadcast_value
17 # Sort the first array
18 sorted_array = np.sort(array1)
19
20 print(indices)
21 print(count)
22 print(broadcast_res)
23 print(sorted_array)
```

Average time0.039 s39.25 msMaximum time0.044 s44.00 ms2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test Case 144msDebug

Expected output

4 1 1 1 2

Value to search: 1

Value to count: 2

Value to add: 2

[0 1 2]

3

Actual output

4 1 1 1 2

Value to search: 1

Value to count: 2

Value to add: 2

[0 1 2]

3

Terminal

Test Cases

Prev

Reset

Submit

Next

Course

Course

New Tab

GitHub Dashboard

View 1 - @pawangalmed123 u X

FPS 57

initaoe.codetantra.com/course.php?eudd=698c54060aba96214c8b7d5e4/contents/698c54060aba96214c8b7d57/6774efaa4ab767eca9da91e

CODETANTRA

Home

Learn Anywhere

202501090044@initaoe.ac.in

Support

Logout

1.2.5. NumPy: Copying and Viewing Arrays

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the original_array and assigning it to view_array.
- Creating a copy of the original_array and assigning it to copy_array.

After completing these steps, observe how modifying the view affects the original_array, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

Original array after modifying view: (original_array)
View array: (view_array)

- After modifying the copy:

Original array after modifying copy: (original_array)
Copy array: (copy_array)

Sample Test Cases

copyAnd...

```
1 import numpy as np
2
3 inputList = list(map(int,input().split(" ")))
4
5 # Original array
6 original_array = np.array(inputList)
7
8 # Create a view
9 view_array = original_array.view()
10
11 # Create a copy
12 copy_array = original_array.copy()
13
14 # Modify the view
15 view_array[0] = 99
16 print("Original array after modifying view:", original_array)
17 print("View array:", view_array)
18
19 # Modify the copy
20 copy_array[1] = 88
21 print("Original array after modifying copy:", original_array)
22 print("Copy array:", copy_array)
23
```

Average time

0.014 s

14.00 ms

Maximum time

0.015 s

15.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

44ms

Debug

Test

Expected output

99 20 30 40 50 60 70 80

Original array after modifying view: [99 20 30 40 50 60 70 80]

View array: [99 20 30 40 50 60 70 80]

Original array after modifying copy: [99 20 30 40 50 60 70 80]

Copy array: [20 88 30 40 50 60 70 80]

Actual output

99 20 30 40 50 60 70 80

Original array after modifying view: [99 20 30 40 50 60 70 80]

View array: [99 20 30 40 50 60 70 80]

Original array after modifying copy: [99 20 30 40 50 60 70 80]

Copy array: [20 88 30 40 50 60 70 80]

Terminal

Test Cases

Prev

Reset

Submit

Next

View recent browsing across windows and devices

Course

New Tab

GitHub Dashboard

View 1 - @pewangalmed123 u X

FPS 96

initaoe.codetantra.com/secure/course.jpg?eudd=698c54060aba96214c8b7d5e4/contents/698c54060aba96214c8b7d57/6774d2cf4ab787eca9da6c0

00%

202301090044@initaoe.ac.in Support Logout

1.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bitwise Operators

You are given two arrays A and B. Your task is to complete the function array_operations, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations:

- Calculate the mean, median, and standard deviation of array A.

3. Bitwise Operations:

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: A, OR B).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Sample Test Cases

different...

```
1 import numpy as np
2
3 def array_operations(A, B):
4
5     # Convert A and B to Numpy arrays
6     A=np.array(A)
7     B=np.array(B)
8
9     # Arithmetic Operations
10    sum_result = A+B
11    diff_result = A-B
12    prod_result = A*B
13
14    # Statistical Operations
15    mean_A = np.mean(A)
16    median_A = np.median(A)
17    std_dev_A = np.std(A)
18
19    # Bitwise Operations
20    and_result = A&B
21    or_result = A|B
22    xor_result = A^B
23
24    # Output results with one space between each element
25    print("Element-wise Sum:", ' '.join(map(str, sum_result)))
26    print("Element-wise Difference:", ' '.join(map(str, diff_result)))
27    print("Element-wise Product:", ' '.join(map(str, prod_result)))
28
29    print("Mean of A: (mean_A)")
30    print("Median of A: (median_A)")
31    print("Standard Deviation of A: (std_dev_A)")
32
33    print("Bitwise AND:", ' '.join(map(str, and_result)))
34    print("Bitwise OR:", ' '.join(map(str, or_result)))
35    print("Bitwise XOR:", ' '.join(map(str, xor_result)))
```

Average time: 0.027 s
27.32 ms

Maximum time: 0.034 s
34.06 ms

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test Case 1

Expected output:
5 7 9 4
5 6 7 8
Element-wise Sum: 6 8 10 12
Element-wise Difference: -4 -6 -8 -8
Element-wise Product: 5 12 21 32
Mean of A: 2.5

Actual output:
5 7 9 4
5 6 7 8
Element-wise Sum: 6 8 10 12
Element-wise Difference: -4 -6 -8 -8
Element-wise Product: 5 12 21 32
Mean of A: 2.5

Terminal

Test Cases

Prev

Next

Submit

Course

Course

New Tab

GitHub Dashboard

View 1 - @penwagmred123 u

FPS 68

initaoe.codedtantra.com/secure/course.php?eudd=698c54060aba96214c8b7d5e4/contents/698c54060aba96214c8b7d5e4/698c54060aba96214c8b7d5e4/6774ec9dfab707eca9da069

80%

202501090044@mitase.ac.in

Support

Logout

1.2.3. Numpy Custom Sequence Generation

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using `numpy` based on these inputs and print the generated sequence.

Input Format:

The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

The program should print the generated sequence based on the input values.

Sample Test Cases

custom S...

```
1 import numpy as np
2
3 # Take user input for the start, stop, and step of the sequence
4 start = int(input())
5 stop = int(input())
6 step = int(input())
7
8 # Generate the sequence using np.arange()# Generate the sequence using np.arange()
9
10 # Print the generated sequence
11 print(np.arange(start, stop, step))
12
```

Average time

0.036 s

36.00 ms

Maximum time

0.044 s

44.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test Case 1

4ms

Debug

Test

Expected output

3

38

2

[1 5 7 9]

Actual output

3

38

2

[1 5 7 9]

Terminal

Test Cases

Prev

Reset

Submit

Next

Course

Course

New Tab

GitHub Dashboard

View 1 - @pewangalmed12's u

FPS 14

initiaze.codedtantra.com/secure/course.php?euid=698c54060aba96214c8b7d5e4/contents/698c54060aba96214c8b7d5e4/6774ee19fab707eca9da7fe

00%

202501090044@mitase.ac.in

Support

Logout

CODETANTRA

Home

Learn Anywhere

1.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

stacking.py

Submit

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- Horizontal Stacking: Stack the two matrices horizontally (side by side).
- Vertical Stacking: Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

stacking.py

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Array1:")
5 arr1 = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Array2:")
8 arr2 = np.array([list(map(int, input().split())) for i in range(3)])
9 # Perform horizontal stacking (hstack)
10
11
12 print("Horizontal Stack:")
13 print(np.hstack((arr1, arr2)))
14 # Perform vertical stacking (vstack)
15 print("Vertical Stack:")
16 print(np.vstack((arr1, arr2)))
17
18
```

Average time

0.069 s

08.75 ms

Maximum time

0.074 s

74.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test Case 1

44ms

Debug

Test

Expected output

Enter Array1:

1 2 3

4 5 6

7 8 9

Enter Array2:

1 2 3

4 5 6

7 8 9

Actual output

Enter Array1:

1 2 3

4 5 6

7 8 9

Enter Array2:

1 2 3

4 5 6

7 8 9

Terminal

Test Cases

Prev

Next

Submit

Reset

Course

Course

New Tab

GitHub Dashboard

View 1 - @pewangalmed12's u X

FPS 66

initaoe.codetantra.com/secure/course.jpg?eudd=698c54060aba96214c8b7d5e4/contents/698c54060aba96214c8b7d55/698c54060aba96214c8b7d57/6774e3ff4ab787eca9da092

CODETANTRA Home Learn Anywhere

202301090044@mitase.ac.in Support Logout

1.2.1. Numpy: Matrix Operations

The given code takes two 3 x 3 matrices, matrix_a, and matrix_b, as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. Addition (matrix_a + matrix_b)

2. Subtraction (matrix_a - matrix_b)

3. Element-wise Multiplication (matrix_a * matrix_b)

4. Matrix Multiplication (matrix_a * matrix_b)

5. Transpose of Matrix A

Input Format:

The user will input 3 rows for matrix_a, each containing 3 integers separated by spaces.

Similarly, the user will input 3 rows for matrix_b, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.

2. The result of Subtraction.

3. The result of Element-wise Multiplication.

4. The result of Matrix Multiplication.

5. The Transpose of Matrix A.

Sample Test Cases

matrixOp...

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Matrix A:")
5 matrix_a = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Matrix B:")
8 matrix_b = np.array([list(map(int, input().split())) for i in range(3)])
9
10 # Addition
11
12 print("Addition (A + B):")
13 print(matrix_a + matrix_b)
14
15 # Subtraction
16 print("Subtraction (A - B):")
17 print(matrix_a - matrix_b)
18
19 # Multiplication (element-wise)
20 print("Element-wise Multiplication (A * B):")
21 print(matrix_a * matrix_b)
22
23 # Matrix multiplication (dot product)
24 print("A dot B:")
25 print(np.dot(matrix_a, matrix_b))
26
27 # Transpose
28 print("Transpose of A:")
29 print(np.transpose(matrix_a))
```

Average time

0.062 s

61.75 ms

Maximum time

0.069 s

69.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test Case 1

Expected output

Enter Matrix A:

1 2 3

4 5 6

7 8 9

Enter Matrix B:

1 1 1

Actual output

Enter Matrix A:

1 2 3

4 5 6

7 8 9

Enter Matrix B:

1 1 1

Terminal

Test Cases

Prev

Reset

Submit

Next

View recent browsing across windows and devices

Course

New Tab

GitHub Dashboard

View 1 - @pewangalmed12's u X

FPS 55

mitasee.codedtantra.com/secure/course.php?euid=698c54060aba96214c8b7d5e4/contents/698c54060aba96214c8b7d5e4/6604440e5d95376955a73c1d

CODETANTRA Home Learn Anywhere

202301090044@mitasee.ac.in Support Logout

1.1.1. Numpy Array Operations

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`.
- The shape of the array using `shape`.
- The total number of elements in the array using `size`.

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, `r` and `c`, representing the number of rows and the number of columns.
- The next `r` lines contain `c` space-separated integers representing the elements of the array, ordered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Note: Use `reshape()` function to reshape the input array with the specified number of rows and columns.

Sample Test Cases

numpyent...

1 import numpy as np # write your code here...
2 rows, columns = map(int, input().split())
3 elements = []
4
5 for i in range(rows):
6 elements_row = map(int, input().split())
7 elements.extend(elements_row)
8
9 matrix = np.array(elements).reshape(rows, columns)
10
11 print(matrix)
12 print(matrix.ndim)
13 print(matrix.shape)
14 print(matrix.size)

Average time: 0.036 s, 36.17 ms | Maximum time: 0.066 s, 66.00 ms | 2 out of 2 shown test case(s) passed, 10 out of 10 hidden test case(s) passed

Test Case 1
Expected output:
[[4
 [1 2 3 4
 [5 6 7 8
 [9 10 11 12
 Actual output:
[[4
 [1 2 3 4
 [5 6 7 8
 [9 10 11 12

Terminal Test Cases

Prev Report Submit Next

◀ Prev Reset Submit Next ▶

Course

Course

New Tab

GitHub Dashboard

View 1 - @pawangakwedi2's u

FPS 81

mitaoa.codedtantra.com/securs/course.jsp?euid=698c54860aba96214c8b7d5a#/contents/698c54860aba96214c8b7d5a/698c54860aba96214c8b7d5a/64ac0cb9b4547106899a4e1b

CODETANTRA Home Learn Anywhere

202501090044@mitaoa.ac.in Support Logout

2.2.1. Linear search Technique

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

- The first line of input contains the array of integers which are separated by space
- The last line of input contains the key element to be searched

Output format:

- If the element is found, print the index. If the element found multiple times, print the starting index
- If the element is not found, print Not found

Sample Test Case:
Input:
1 2 3 4 5 6
3
Output:
2

Sample Test Cases

CTP1708...

```
1 arr = list(map(int, input().split()))
2
3 # Read key element
4 key = int(input())
5
6 found = False
7
8 # Linear search
9 for i in range(len(arr)):
10     if arr[i] == key:
11         print(i)
12         found = True
13         break
14
15 if not found:
16     print("Not found")
```

Average time0.023 s22.75 msMaximum time0.028 s30.00 ms

2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1

Expected output1 2 3 4 5 63

Actual output1 2 3 4 5 63

Test case 2

Expected output

Actual output

TerminalTest cases

PrevResetSubmitNext

Course

Course

New Tab

GitHub Dashboard

View 1 - @pawangalmed123 u X

FPS 43

mitasee.codedtantra.com/secure/course.jpg?eudd=698c54060aba96214c8b7d5e4/contents/698c54060aba96214c8b7d52/698c54060aba96214c8b7d53/697335a7e2971206fe82d6a0

00%

202301090044@mitasee.ac.in Support Logout

3.1.2. Dictionary Operations

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:
1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:
1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.
The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Note:

- All operations must be performed using dictionary methods.
- Perform deletion only if possible; leave the dictionary unchanged.
- Refer to the visible test cases for better understanding and strictly match with the input/outputs.

Sample Test Cases

DictOper...

```
1 # Initial dictionary with 10 predefined records
2 student = {
3     1: "Amit",
4     2: "Riya",
5     3: "Kiran",
6     4: "Neha",
7     5: "Arjun",
8     6: "Pooja",
9     7: "Rahul",
10    8: "Sneha",
11    9: "Vikram",
12    10: "Anjali"
13 }
14
15 # write your code here...# write your code here...
16
17 print("Original Dictionary:", student)
18 key = int(input())
19 value = input()
20 student[key] = value
21 print("After Insertion:", student)
22 key = int(input())
23 value = input()
24 student[key] = value
25 print("After Update:", student)
26 key = int(input())
27 student.pop(key, None)
28 print("After Deletion:", student)
29 print("Traversing Dictionary:")
30 for key, value in student.items():
31     print(f"{key} : {value}")
32
33
34
```

Average time0.067 s06.75 msMaximum time0.072 s72.00 ms

2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1

Expected outputOriginal Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}11SureshAfter Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}12

Actual outputOriginal Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}11SureshAfter Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}12

Terminal

Test Cases

Prev

Next

Submit

The screenshot displays a web browser window with multiple tabs. The active tab is titled "View 1: @psengaiwad12's untitled project" and shows a coding exercise on "Dictionary Operations".

The exercise instructions state: "Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program."

The "Operations to be Performed:" section lists four tasks:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

The "Input and Output Format:" section provides details for each operation:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

A "Note:" section specifies:

- All operations must be performed using dictionary methods.
- Perform deletion only if possible; leave the dictionary unchanged.
- Refer to the visible test cases for better understanding and strictly match with the input/outputs.

The "Sample Test Cases" section shows a table with 4 test cases, all of which are "Passed".

The code editor displays the following Python code:

```
1 # Dictionary Operations
2 # Predefined records
3 student = {
4     1: "Adit",
5     2: "Kya",
6     3: "Kiran",
7     4: "Neha",
8     5: "Arjun",
9     6: "Pooja",
10    7: "Rahul",
11    8: "Sneha",
12    9: "Vikram",
13    10: "Anjali"
14 }
15
16 # write your code here...# write your code here...
17
18 print("Original Dictionary:", student)
19 key = int(input())
20 value = input()
21 student[key] = value
22 print("After Insertion:", student)
23 key = int(input())
24 value = input()
25 student[key] = value
26 print("After Update:", student)
27 key = int(input())
28 student.pop(key, None)
29 print("After Deletion:", student)
30 print("Traversing Dictionary:")
31 for key, value in student.items():
32     print(f"{key} : {value}")
33
34
```

The code is tested, showing 2 out of 2 test cases passed. The test case details show the expected output and the actual output, which match perfectly.

Course

Course

New Tab

GitHub Dashboard

View 1 - @pawargakwad12's u

FPS 44

initaoe.codedtantra.com/secure/course.jsp?eucld=698c54860aba96214c8b7d5ef/contents/698c54860aba96214c8b7d62/698c54860aba96214c8b7d63/667bbd7f010c0fd690f3f83

80%

Support Logout

CODETANTRA

Home Learn Anywhere

20250100044@mitaoe.ac.in

2.1.1. List operations

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:
1. Add
2. Remove
3. Display
4. Quit
The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:
• Add: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
• Remove: Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
• Display: Displays the current list of integers. If the list is empty, display "List is empty".
• Quit: Exits the program.
• The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Sample Test Cases

listOps.py

```
1 # write the code..# write the code..
2 lst = []
3
4 while True:
5     print("1. Add")
6     print("2. Remove")
7     print("3. Display")
8     print("4. Quit")
9
10    choice = int(input("Enter choice: "))
11
12    if choice == 1:
13        num = int(input("Integers: "))
14        lst.append(num)
15        print("List after adding:", lst)
16
17    elif choice == 2:
18        if len(lst) == 0:
19            print("List is empty")
20        else:
21            num = int(input("Integers: "))
22            if num in lst:
23                lst.remove(num)
24                print("List after removing:", lst)
25            else:
26                print("Element not found")
27
28    elif choice == 3:
29        if len(lst) == 0:
30            print("List is empty")
31        else:
32            print(lst)
33
34    elif choice == 4:
```

0.219 s
219.33 ms

0.304 s
308.00 ms

2 out of 2 shown test case(s) passed
5 out of 1 hidden test case(s) passed

Test case 1

Expected output

Actual output

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Enter choice: 1

Integers: 11

Integers: 11

Terminal

Test cases

Course

Course

New Tab

GitHub Dashboard

View 1 · @pavangokwad123

FPS 49

mitace.codedtantra.com/secure/course.jsp?ecld=698c54060aba96214c8b7d5e4/content/s/698c54060aba96214c8b7d51/6774cc89f4ab7b7eca9d9f99

60%

202501090044@mitace.ac.in Support Logout

1.2.4. Pattern - 2

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern

Sample Test Cases

numberP...

```
1 # Type Content here...# Type Content here...
2 n = int(input())
3 for i in range(1, n + 1):
4     for j in range(1, i + 1):
5         print(j, end=" ")
6     print()
7
8
9
```

Average time0.013 s12.50 ms

Minimum time0.014 s14.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 14 msDebug

Expected output	Actual output
1	1
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5

Terminal

Test cases

PrevNextSubmitSolveNext

Course

Course

New Tab

Gitlab Dashboard

View 1 - @pawankwad12's u X

FPS 94

mitaoe.codedtantra.com/secure/course.jsp?eudd~698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d5f/698c54860aba96214c8b7d61/6774c521f4ab787eca9d0a75

CODETANTRA Home Learn Anywhere

202501090044@mitaoe.ac.in Support Logout

1.2.3. Pattern - 1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases

rightangl...

```
1 #Type Content here...
2 n = int(input())
3
4 for i in range(1, n + 1):
5     for j in range(i):
6         print('*', end=' ')
7     print()
```

Average time0.015 s14.83 msMaximum time0.020 s20.00 ms

2 out of 2 shown test case(s) passed4 out of 4 hidden test case(s) passed

Test case 1

Expected output

Actual output

TerminalTest cases

Prev

Reset

Submit

Next

View recent browsing across windows and devices

Course

New Tab

GitHub Dashboard

View 1 - @penwagalmv2's u

FPS 15

initiaze.codetantra.com/secure/course.jpg?eucdd=696c54060aba96214c8b7d5e#/contents/696c54060aba96214c8b7d5f/696c54060aba96214c8b7d61/6602597fa290d347a73c1700

80%

202301090044@mitase.ac.in

Support

Logout

CODETANTRA

Home

Learn Anywhere

1.2.2. Fibonacci Series

Solve

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

- A single integer n, representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases

fibonacci...

```
1 def fibonacci(n):
2
3     # write the code..
4     if n <= 1:
5         return 0
6     if n==2:
7         return 1
8     else:
9
10        return fibonacci(n-1) + fibonacci(n-2)
11
12
13
14 n = int(input())
15 for i in range(1, n + 1):
16     print(fibonacci(i), end=" ")
17
```

Average time: 0.034 s, 33.00 ms

Maximum time: 0.082 s, 82.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1: 43ms

Expected output: 0 1 1 2 3

Actual output: 0 1 1 2 3

Test case 2: 43ms

Terminal

Test cases

Prev

Reset

Submit

Next

Course

Course

New Tab

GitLab Dashboard

View 1 - @pavangalnad12's u

FPS 86

mitaoe.codetantra.com/secure/course.jsp?euid=698c54860aba96214c8b7d5e*/contents/698c54860aba96214c8b7d5f/698c54860aba96214c8b7d61/6773f211f4ab787eca3d15d0

202301090044@mitaoe.ac.in

Support

Logout

12.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses. The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n, the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Notes:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

Sample Test Cases

passorfa...

```
1 # write your code here# write your code here
2 n = int(input())
3
4 marks = list(map(int, input().split()))
5
6 if len(marks) != n:
7     print("Invalid input")
8 elif any(m < 40 for m in marks):
9     print("Fail")
10 else:
11     percentage = sum(marks) / n
12     print(f"Aggregate Percentage: {percentage:.2f}")
13
14     if percentage > 75:
15         grade = "Distinction"
16     elif percentage >= 60:
17         grade = "First Division"
18     elif percentage >= 50:
19         grade = "Second Division"
20     else:
21         grade = "Third Division"
22     print(f"Grade: {grade}")
23
```

Average time0.022 s22.30 msMaximum time0.027 s27.00 ms

5 out of 5 shown test case(s) passed5 out of 5 hidden test case(s) passed

Test case 1

Expected output

Actual output

55.05 28.8955.05 28.89

Aggregate Percentage: 71.75Aggregate Percentage: 71.75

Grade: First DivisionGrade: First Division

Test cases

Terminal

Test cases

Course

Course

New Tab

GitHub Dashboard

View 1 · @pawengained12's u X

FPS 81

mitaoe.codetantra.com/secure/course.jsp?euid=690c54060aba96214c8b7d5e4/contents/690c54060aba96214c8b7d5d/690c54060aba96214c8b7d60/670dd2f0fa4f5e20f6fcd1a

CODETANTRA Home Learn Anywhere

202501090144@mitaoe.ac.in Support Logout

1.1.5. Multiplication Table

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

```
<number> x <multiplier> = <result>
```

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

Sample Test Cases

multiplica...

3 num = int(input())
2 for i in range(1,11):
3 print(f"{num} x {i} = {num*i}")
4
5

Average time: 0.017 s
Maximum time: 0.019 s
17.25 ms 10.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 48ms Debug

Expected output
8 x 1 = 8
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40

Actual output
8 x 1 = 8
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40

Terminal Test cases

< Prev Next >

Course

Course

New Tab

GitHub Dashboard

View 1 - @penwagalmad12's u

FPS 90

mitaoe.codetantra.com/secure/course.jpg?eucdd=698c54060aba96214c8b7d5e4/contents/698c54060aba96214c8b7d5e4/698c54060aba96214c8b7d5e4/6774c276f4ab7878ca9d88e3

00%

Support Logout

1.1.4. Reverse a Number

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

The input is a single integer

Output Format:

The output prints a single integer, which is the reversed number.

Sample Test Cases

reversen...

```
1 #write your code here...#write your code here...
2 num = int(input())
3 reversed_num = int(str(num)[::-1])
4 print(reversed_num)
```

Average time: 0.012 s, 12.20 ms

Maximum time: 0.014 s, 14.00 ms

2 out of 2 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 44ms

Expected output: 5380

Actual output: 5380

Test case 2 42ms

Terminal

Test cases

Prev

Report

Submit

Next

Course

Course

New Tab

GitHub Dashboard

View 1 - @pawangakwad12's

FPS

mitaoe.codetantra.com/secure/course.jsp?euid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d5e/66ab8cfc4a63741f9be3268c

CODETANTRA Home Learn Anywhere

202501090144@mitaoe.ac.in Support

1.1.3. Days between Two Dates

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format `YYYY-MM-DD`
- The second line contains the second date in the format `YYYY-MM-DD`

Output Format:

- An Integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

Sample Test Cases

noCMay...

```
1 from datetime import date
2
3 #write your code here...
4 from datetime import datetime
5 date1 = input()
6 date2 = input()
7 D1=datetime.strptime(date1,"%Y-%m-%d")
8 D2=datetime.strptime(date2,"%Y-%m-%d")
9 days=(D2-D1).days
10 print(days)
```

Actual execution time: 0.030 s

Maximum execution time: 0.064 s

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

Expected output

Actual output

2018-07-02

2018-07-02

2018-11-02

2018-11-02

121

121

Test case 2

Expected output

Actual output

2018-07-02

2018-07-02

2018-11-02

2018-11-02

121

121

Terminal

Test cases

CourseNew TabGitHub DashboardView 1 - @pawangalnvad12's v: XFPS 30

mitaoe.codetantra.com/secure/course.jsp?eucdd=638c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d60/6773eb5d4ab787eca9d0bf7

CODETANTRAHomeLearn Anywhere202501090444@mitaoe.ac.inSupportLogout

1.1.2- Conditional Calculation Based on the Number of Digits

Write a Python program that accepts an integer n as input. Depending on the number of digits in n .

Constraints:
 $1 \leq n \leq 999$

Input Format:
• The input consists of a single integer n .

Output Format:
• If n is a single-digit number, print its square.
• If n is a two-digit number, print its square root (rounded to two decimal places).
• If n is a three-digit number, print its cube root (rounded to two decimal places).
• Else print "Invalid".

Sample Test Cases

condition...

```
1 #write your code here...#write your code here...
2 import math
3 number = int(input())
4 if 1 <= number <= 9:
5     print(number**2)
6 elif 10 <= number <= 99:
7     print(f"{math.sqrt(number):.2f}")
8 elif 100 <= number <= 999:
9     print(f"{math.cbrt(number):.2f}")
10 else:
11     print("Invalid")
```

Average time
0.013 s
11.83 ms

Maximum time
0.015 s
18.00 ms

4 out of 4 shown test case(s) passed
3 out of 3 hidden test case(s) passed

Test case 1
Expected output
11

Actual output
9

Test case 2

Test case 3

Terminal

Test cases

Prev

Reset

Submit

Next

Course

Course

New Tab

GitHub Dashboard

View 1 - @penwagalmad12's u

FPS 93

mitaoe.codetantra.com/secure/course.jpg?eudd=698c54060aba96214c8b7d5e4/contents/698c54060aba96214c8b7d5d/698c54060aba96214c8b7d60/6773e9c2f4ab787eca9d09e4

00%

202501090044@mitaoe.ac.in

Support

Logout

1.1.1. Calculate Momentum

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula:
$$p = m \times v$$
where:
 m is the mass of the object (in kilograms).
 v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Sample Test Cases

calculate...

```
1 #Type Content here...
2 mass=float(input())
3
4 velocity = float(input())
5
6 momentum =mass*velocity..
7
8 print(f"{momentum:.2f}%gm/s")
```

Average time: 0.021 s, 20.00 ms

Maximum time: 0.025 s, 20.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 20ms

Expected output: 5.0, 36.0

Actual output: 5.0, 36.0

Test case 2 20ms

Terminal

Test cases

Prev

Reset

Submit

Next